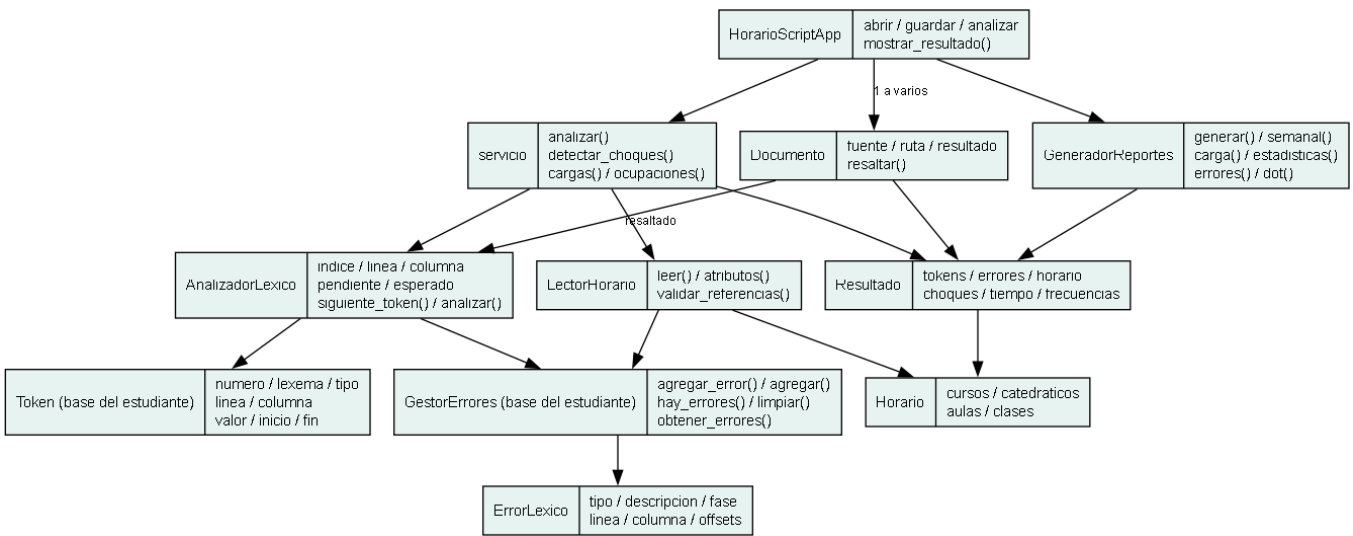


Manual técnico

Proyecto 1 · Versión 1.0.0 · Segundo semestre 2026

HorarioScript convierte un archivo .hor en tokens, diagnósticos y un modelo de horario. El analizador se implementa manualmente: no importa re ni usa split/find para tokenizar. Python 3.10+ y Tkinter son los únicos requisitos de ejecución; Graphviz es opcional al renderizar el DOT.

1. Arquitectura y clases



Componente	Responsabilidad
Token y GestorErrores	Clases del primer commit del estudiante, ampliadas con valor, offsets y fase; mantienen sus métodos originales.
AnalizadorLexico	Recorre caracteres, conserva lexemas y produce tokens o errores mediante siguiente_token().
LectorHorario	Valida bloques, atributos y referencias; construye solo registros utilizables. No mezcla estructura con errores léxicos.
Servicio y modelos	Coordina fases; detecta choques, suma carga y calcula unión de intervalos de ocupación.
GUI y reportes	Cada Documento conserva su análisis; GeneradorReportes produce cuatro HTML, DOT, CSV y JSON.

2. AFD completo de reconocimiento

Estado inicial I. Los nueve estados del bucle son I, PAL, NUM, HORA, MALNUM, TEXTO, ESCAPE, HASH y COMENTARIO. A y E representan salidas del método, no estados persistentes. Una nueva llamada reinicia I sin perder el cursor.

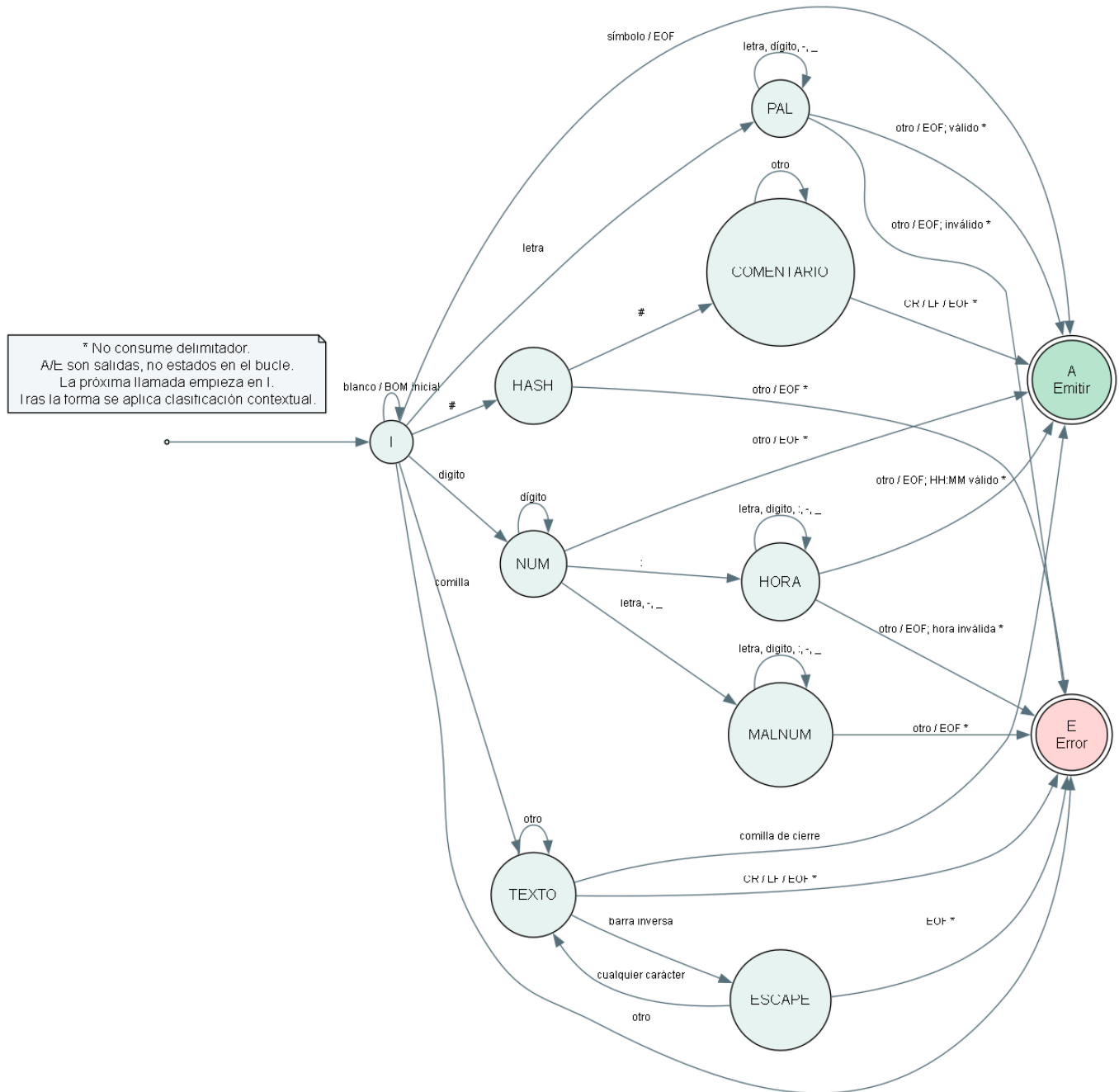


Diagrama generado con Graphviz desde docs/diagramas/afd.dot. Las flechas con * no consumen el delimitador; por eso la siguiente llamada lo procesa correctamente.

3. Tabla de transiciones · parte 1

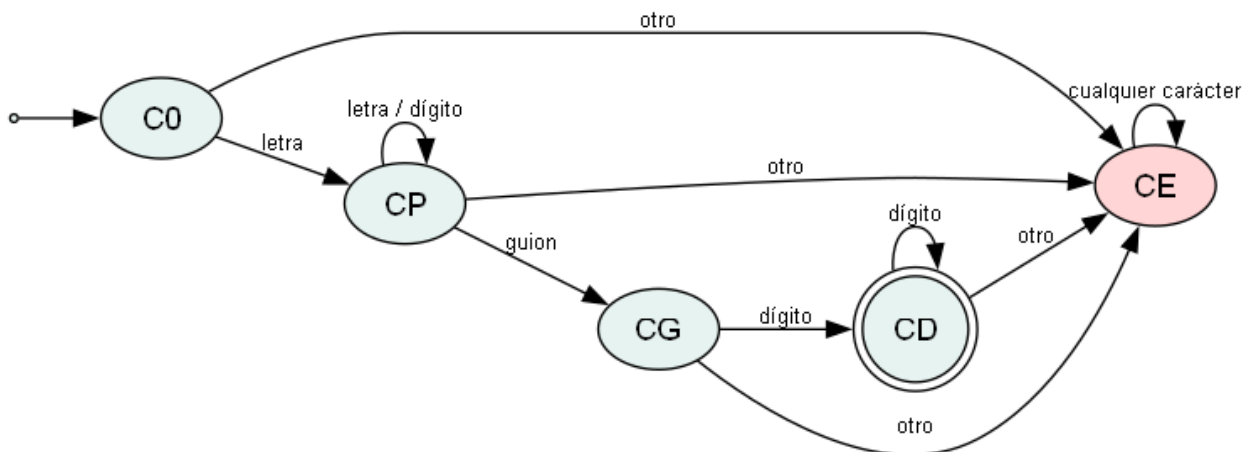
Cada grupo de entradas de un estado es disjunto. “Otro” excluye los grupos anteriores del mismo estado. CRLF se consume como una nueva línea lógica.

Estado	Entrada	Destino	Acción
I	Espacio, tab, CR, LF; BOM solo inicial	I	Consumir; actualizar posición
I	Letra ASCII	PAL	Consumir inicio de palabra
I	Dígito ASCII	NUM	Consumir inicio numérico
I	Comilla doble	TEXTO	Consumir apertura; valor vacío
I	#	HASH	Consumir primer #
I	{ } [] : , ;	A	Consumir y emitir SIMBOLO
I	EOF	A	Emitir EOF; no consumir
I	Cualquier otro carácter	E	Consumir uno; CARACTER_NO_RECONOCIDO
PAL	Letra, dígito, guion o _	PAL	Consumir secuencia completa
PAL	Otro carácter o EOF	A / E	No consumir delimitador; reservada, código o error
NUM	Dígito	NUM	Consumir
NUM	:	HORA	Consumir
NUM	Letra, guion o _	MALNUM	Consumir; candidato numérico inválido
NUM	Otro carácter o EOF	A	No consumir delimitador; emitir ENTERO
HORA	Letra, dígito, : guion o _	HORA	Consumir candidato completo
HORA	Otro carácter o EOF	A / E	Validar HH:MM y rango; HORA o HORA_FUERA_DE_RANGO
MALNUM	Letra, dígito, : guion o _	MALNUM	Consumir candidato completo
MALNUM	Otro carácter o EOF	E	No consumir; CODIGO_MAL_FORMADO

3. Tabla de transiciones · parte 2

Estado	Entrada	Destino	Acción
TEXTO	Comilla doble	A	Consumir cierre; CADENA o CODIGO
TEXTO	Barra inversa	ESCAPE	Consumir escape
TEXTO	CR, LF o EOF	E	No consumir; CADENA_SIN_CERRAR
TEXTO	Cualquier otro carácter	TEXTO	Consumir y añadir al valor
ESCAPE	EOF	E	CADENA_SIN_CERRAR
ESCAPE	CR o LF	TEXTO	Consumir continuación; no añadir al valor
ESCAPE	Cualquier otro carácter	TEXTO	Consumir; n/t/r se decodifican, resto literal
HASH	#	COMENTARIO	Consumir segundo #
HASH	Otro carácter o EOF	E	No consumir; CARACTER_NO_RECONOCIDO
COMENTARIO	CR, LF o EOF	A	No consumir; emitir COMENTARIO_LINEA
COMENTARIO	Cualquier otro carácter	COMENTARIO	Consumir sin validar contenido

4. Sub-AFD de códigos



Acepta únicamente si termina en CD. CP acumula el prefijo; CG exige al menos un dígito después del guion. CE es rechazo absorbente: el código retorna False inmediatamente. Al llegar a EOF en C0, CP o CG se rechaza; en CD se acepta.

5. Clasificación, prioridades y posiciones

Palabras y códigos

Se consume la secuencia completa y se compara con tablas de igualdad exacta: bloques, elementos, relaciones, atributos, días y categorías. Si no coincide, se aplica el sub-AFD de códigos. La entrada HORARIO1 nunca se divide artificialmente en HORARIO + 1.

Ambigüedad del ejemplo oficial

La regla escrita dice letras-guion-dígitos, pero BD2-0812 figura como ejemplo válido. Se admite una letra inicial seguida por letras o dígitos antes del guion. Los códigos se aceptan con comillas, como en el ejemplo, o sin ellas, como permiten las reglas de prioridad. Un prefijo iniciado en dígito se rechaza.

Formas y contexto finito

El AFD identifica formas. `_emitir` conserva pendiente/esperado para los valores de código, aula, clase, con, en, día, inicio, fin y categoría. Los comentarios no alteran ese contexto. Así un nombre entre comillas sigue siendo CADENA, mientras que un valor inválido después de código: genera CODIGO_MAL_FORMADO. No se atribuyen transiciones sintácticas completas al AFD.

Horas

HORA acumula el candidato completo, incluso caracteres erróneos. `minutos_validos` exige longitud 5, dos dígitos, dos puntos y otros dos dígitos; calcula $h*60+m$, exige $m<60$ y $360 \leq \text{total} \leq 1260$. La forma y el rango se validan después del reconocimiento del candidato. No se admiten horas con comillas.

Cadenas y comentarios

Se conservan comillas y escapes en lexema y se decodifica valor. Se admiten `\` y `\\`, `n/t/r` escapados y una nueva línea física escapada como continuación. Otros escapes producen el carácter literal. Una cadena abierta termina en error al encontrar CR/LF no escapado o EOF. Un comentario `##` es válido hasta CR, LF o EOF: el caso de "comentario sin cerrar" del enunciado no puede ser un error con esta sintaxis.

Línea, columna y offsets

Línea y columna comienzan en 1. LF, CR o CRLF cuentan una nueva línea; una tabulación avanza al siguiente tope de cuatro columnas. Un BOM inicial no ocupa columna. Los offsets inicio/fin son índices de caracteres de Python, no de bytes. El editor normaliza saltos a LF; la lectura por consola conserva CRLF. El resaltado y la selección usan offsets sobre el texto que se analizó.

6. Recuperación, extracción y choques

Recuperación léxica

Un carácter desconocido consume exactamente un carácter. Un candidato de palabra/hora inválido consume su unidad completa hasta un delimitador; una cadena abierta se detiene antes del salto. El método devuelve ERROR_LEXICO interno, agrega un diagnóstico y permite la siguiente llamada. Los tokens válidos tienen numeración consecutiva; EOF y errores no aparecen como tokens válidos.

Extracción estructural

El lector auxiliar exige HORARIO y las cuatro secciones exactamente una vez, en cualquier orden. Permite coma final entre elementos y sección de clase opcional; exige los atributos requeridos y los separadores. Recupera en un siguiente elemento, sección o cierre. Se rechazan códigos duplicados, valores no positivos, intervalos invertidos y referencias inexistentes. Los catálogos vacíos son válidos.

Choques

Se agrupa por día y se ordena por inicio. Antes de cada clase se retiran las activas cuyo $\text{fin} \leq \text{inicio actual}$. Para cada activa restante se compara docente y aula. El traslape es $[\max(\text{inicios}), \min(\text{finales})]$; se guarda un solo par con uno o ambos motivos. Se compara entre todas las secciones. Dos clases que solo comparten sección no constituyen choque según el criterio obligatorio.

Carga y ocupación

Carga docente: suma de fin-inicio, incluso si hay traslapes; cursos y secciones son conjuntos distintos. Umbrales continuos: BAJA 0-4 h, NORMAL >4-10 h, ALTA >10-15 h y SATURADA >15 h, para evitar huecos en horas fraccionarias. Ocupación de aula: longitud de la unión de intervalos por día, dividida entre $6 \cdot 15 \cdot 60 = 5400$ minutos disponibles. Más del 80% se pinta rojo. Los empates se muestran completos y sin clases se indica Sin asignaciones.

Costo y límites

El reconocimiento examina caracteres secuencialmente; mantiene tokens y fuente en memoria. La detección cuesta $O(n \log n + n \cdot a)$, donde a es el máximo de clases activas; en el peor caso es $O(n^2)$, y puede producir esa cantidad de pares. El resaltado automático se limita a 200 000 caracteres, pero F5 analiza el documento completo. La GUI trabaja en el hilo principal: entradas extremadamente grandes pueden bloquearla durante el cálculo. No se afirma una capacidad ilimitada.

7. Reportes, trazabilidad y reproducción

Los HTML incluyen su propio CSS, codificación UTF-8 y enlaces relativos entre los cuatro reportes. Toda entrada del usuario se escapa antes de insertarla en HTML. Los identificadores del DOT son internos; los textos se serializan entre comillas. Se llama a Graphviz con una lista de argumentos, sin shell.

Al editar un documento se invalidan sus resultados y se desactivan los botones de reportes hasta analizar de nuevo. Cada análisis genera una carpeta con marca de tiempo; los reportes de demostración entregados son independientes de las ejecuciones posteriores.

Se conserva el commit original 86118a7 y las clases de William. token.py se trasladó al paquete para no ocultar el módulo estándar de Python. Se preservan __str__, agregar_error, hay_errores, limpiar y obtener_errores. Los nuevos commits corresponden al desarrollo real asistido por IA.

Validación reproducible

```
python -m unittest discover -s tests -v
python herramientas/ejecutar_casos.py
python main.py --analizar entrada/01_valido.hor
python herramientas/diagramas.py dot
```

44 pruebas automatizadas; los doce casos y sus resultados están en evidencia_casos.json. Se comprueban todos los minutos del día, traslapes mediante un oráculo de conjuntos, escape de HTML, exportaciones, referencias, posiciones y recuperación. El tiempo registrado es una medición local, no una garantía de rendimiento.

Fuentes

Enunciado HorarioScript_Proyecto1.pdf, secciones 4.2-4.8 y rúbrica 8.2. Python/Tkinter: <https://docs.python.org/3/library/tkinter.html>. Graphviz/DOT: <https://graphviz.org/doc/info/lang.html>. Consultadas para las herramientas; el diseño y el código del AFD se escribieron manualmente.

Tutor y repositorio

Sección académica B+, tutor SamuelAguilar18. Se conserva el nombre LFP_B-_2S2026_202210198 por decisión del estudiante; no cambia su sección académica. El enunciado exige repositorio privado; comprobar esta configuración antes de entregar.